

Plan Vector — 1 mois jour par jour

Plan détaillé pour les 4 premières semaines de ton stage Welyne × AI Commandos.

Conçu pour qu'on travaille **ensemble** à chaque étape : chaque "session" correspond à un échange avec moi.

Vue d'ensemble

Semaine	Thème	À la fin tu auras...
1	Setup pro + premiers endpoints + OpenAI	Vector qui répond à des questions simples sur un CSV
2	Agent intelligent (function calling)	Vector qui appelle des fonctions Python tout seul
3	Base de données + RAG	Vector connecté à Postgres + pgvector
4	Détection d'insights + démo	Vector qui découvre les patterns cachés

Charge estimée : ~30-35 h/semaine de travail effectif (6-7h/jour, du lundi au vendredi, avec pauses). Le weekend est **off** par défaut.

Comment on travaille ensemble

À chaque "session" mentionnée :

1. **Tu ouvres un chat avec moi** (ou tu continues celui-ci)
2. **Tu me dis sur quoi tu veux bosser** (avec le contexte : "je suis sur le jour X, on devait faire Y")
3. **On code ensemble** : je t'explique, tu copies/comprends, on debug
4. **Tu reviens si tu coinces** — ne reste jamais bloquée plus d'1h seule

Règles de survie pour le stage

-  **Pauses régulières** : 5 min toutes les heures, 1h le midi
-  **Pas plus de 23h** : le sommeil consolide ce que tu apprends
-  **Journal quotidien** : 5 min en fin de journée pour noter ce que tu as fait/appris
-  **Maximum 1h bloquée seule** : après → tu reviens vers moi ou ton encadrant
-  **Done is better than perfect** : un code qui marche aujourd'hui > un code parfait dans 3 semaines

⚠ Important : ce plan s'adapte

Si ton kick-off avec ton encadrant révèle des choix techniques différents (framework agent différent, scope différent, frontend hors périmètre...), **on ajustera**. Ce plan est un **template solide**, pas un dogme. Le plus important :

- Si Welyne te donne un cahier des charges précis avec un planning à eux → **leur plan prend le dessus**, le nôtre devient ton entraînement perso
 - Si Welyne te laisse autonome → ce plan te sert de feuille de route
-

📅 SEMAINE 1 — Fondations & premier agent

Thème : Setup propre + maîtrise FastAPI + premier appel OpenAI

Jour 1 — Lundi : Kick-off + récupération des accès

🎯 Objectif du jour

Sortir de la journée avec une **vision claire du périmètre** et tous les accès nécessaires pour démarrer.

🛠 Tâches concrètes

Matin (3-4h) :

- ☐ Arrivée chez Welyne, présentations équipe
- ☐ **Réunion de cadrage 60 min** avec ton encadrant (avec ta checklist sortie il y a quelques jours)
- ☐ Prendre des notes structurées dans `notes_kickoff.md`

Après-midi (3-4h) :

- ☐ Réception/création des accès :
- ☐ Email Welyne pro
- ☐ Slack/Teams
- ☐ GitHub du repo (clone-le)
- ☐ Notion/Jira pour le suivi de tâches
- ☐ Clé API OpenAI/Azure OpenAI pro
- ☐ Lire le README du repo Welyne (s'il existe) + toute doc fournie
- ☐ Cloner le repo dans `~/projects/`
- ☐ Envoyer un compte-rendu écrit du cadrage à ton encadrant (WhatsApp ou email)

💛 Session avec moi

Une fois le kick-off fait → tu m'envoies tes notes ou un résumé. On analyse ensemble si le scope est réaliste et on adapte le plan si nécessaire.

✓ Tu finis la journée avec

- Compte-rendu de kick-off envoyé
 - Tous les accès en main
 - Repo cloné en local
 - Le plan ajusté selon les réponses du kick-off
-

Jour 2 — Mardi : Setup environnement projet + FastAPI propre

🎯 Objectif du jour

Avoir une structure de code propre, prête à grandir, avec gestion des variables d'environnement et un premier endpoint qui marche.

📚 Concepts à comprendre

- Structure d'un projet Python pro (qu'est-ce que c'est qu'un "package")
- Variables d'environnement et fichier `.env`
- Pydantic Settings pour gérer la config proprement

🔧 Tâches concrètes

Matin : structure du projet

■ Créer la vraie structure :

```
vector-poc/  
├── app/  
│   ├── __init__.py  
│   ├── main.py  
│   ├── config.py      ← gestion des settings  
│   ├── models/        ← Pydantic models  
│   ├── routers/       ← endpoints organisés  
│   └── services/      ← logique métier  
├── tests/  
├── .env                ← variables secrètes (NEVER commit)  
├── .env.example        ← template public  
├── .gitignore  
├── README.md  
└── requirements.txt
```

- Mettre en place `.gitignore` propre
- Créer `.env.example` (template) et `.env` (vraies clés)

Après-midi : refactoriser le POC d'hier

- Migrer `main.py` vers cette structure
- Endpoints `/` et `/health` dans un router séparé
- Endpoint `/analyze` dans un router avec service séparé
- Premier commit Git propre

🧡 Session avec moi

Session #1 — "Structurer un projet FastAPI pro" : on fait ça ensemble, je te montre les conventions, on refactorise ton code étape par étape.

✅ Tu finis la journée avec

- Code organisé en modules
 - `.env` qui fonctionne (test : print d'une variable depuis ton code)
 - Premier commit sur ton repo de POC
 - Tu comprends pourquoi cette structure (et pas tout dans `main.py`)
-

Jour 3 — Mercredi : Premier appel OpenAI depuis FastAPI

🎯 Objectif du jour

Vector devient "intelligent" : il prend une question en français et y répond en utilisant GPT-4o-mini.

📚 Concepts à comprendre

- Comment marche l'API OpenAI (modèles, messages, tokens, coût)
- La différence `system`, `user`, `assistant` dans les messages
- Comptage de tokens avec `tiktoken`
- Coût d'un appel (~\$0.001 pour les petits prompts en gpt-4o-mini)

🛠️ Tâches concrètes

Matin : premier appel OpenAI

- Endpoint `POST /ask` qui prend `{"question": "..."}` en entrée
- Appel à OpenAI avec un system prompt initial
- Renvoyer la réponse formatée

Après-midi : Vector + contexte CSV

- Endpoint `POST /ask-about-csv` qui prend une question ET un CSV
- Injecter un résumé du CSV (head, stats) dans le prompt OpenAI
- Vector répond en utilisant ce contexte
- Test : *"Combien de clients dans le segment Premium ?"*

🧡 Session avec moi

Session #2 — "Brancher OpenAI dans FastAPI proprement" : on code le service OpenAI avec retry, gestion d'erreurs, comptage de tokens.

✅ Tu finis la journée avec

- Vector qui répond en langage naturel
 - Comprend ce que c'est qu'un prompt système
 - Premier appel API qui te coûte 0.001\$ — magique
-

Jour 4 — Jeudi : Pydantic + outputs structurés

🎯 Objectif du jour

Vector ne renvoie plus du texte libre, mais des **données structurées** (JSON) — utilisable par CORE ou un frontend.

📚 Concepts à comprendre

- Pourquoi les sorties structurées sont critiques pour un agent
- Pydantic models comme contrat d'interface
- `response_format={"type": "json_object"}` d'OpenAI
- (Optionnel) Le package `instructor` qui simplifie tout

🛠️ Tâches concrètes

Matin : Pydantic models

- ▢ Définir des models précis : `AnalysisResult`, `ClientInsight`, `Recommendation`, etc.
- ▢ Vector génère un JSON conforme à ces models
- ▢ Gestion des erreurs si OpenAI renvoie un JSON malformé

Après-midi : endpoint complet d'analyse

- ▢ `POST /analyze-and-insight` : prend un CSV, renvoie un objet `AnalysisResult` avec stats + 3-5 insights LLM
- ▢ Tester avec `clients.csv` du dataset synthétique
- ▢ Tester avec `sales.csv`

🧡 Session avec moi

Session #3 — "Outputs structurés et fiables" : on voit ensemble comment forcer OpenAI à renvoyer du JSON propre, gérer les erreurs de parsing, valider avec Pydantic.

✓ Tu finis la journée avec

- Vector qui renvoie du JSON structuré garanti
 - Tes premiers vrais Pydantic models métier
 - Une appréciation de POURQUOI Pydantic est partout en FastAPI
-

Jour 5 — Vendredi : Polish + Git workflow + premier PR

🎯 Objectif du jour

Code propre, documenté, testable. Première Pull Request si tu travailles sur le repo Welyne.

🔧 Tâches concrètes

Matin : qualité de code

- ❑ Installer Ruff (`pip install ruff`) et lancer `ruff check .` puis `ruff format .`
- ❑ Type hints partout (`mypy app/`) pour vérifier
- ❑ Docstrings sur toutes les fonctions importantes

Après-midi : workflow Git

- ❑ Créer une branche `feat/csv-analysis`
- ❑ Commits suivant Conventional Commits
- ❑ Push + créer une Pull Request si repo Welyne
- ❑ Demander code review à ton encadrant
- ❑ Écrire ton **journal de bord semaine 1** : ce que tu as fait, appris, questions ouvertes

💛 Session avec moi

Session #4 — "Hygiène code : linting, types, commits" : on revoit ensemble la qualité de ton code et on met en place les bonnes habitudes.

✓ Tu finis la semaine avec

- ✓ Vector POC fonctionnel sur ton PC
- ✓ Structure de projet pro
- ✓ Appels OpenAI maîtrisés
- ✓ Outputs JSON structurés
- ✓ Code propre, première PR
- ✓ Compte-rendu hebdo écrit

🎉 Tu peux te détendre tout le weekend, tu l'as mérité

SEMAINE 2 — L'agent qui raisonne (Function Calling)

Thème : Transformer Vector en VRAI agent qui décide quelles fonctions appeler.

Jour 6 — Lundi : Comprendre le Function Calling

Objectif du jour

Comprendre LE concept central des agents IA. Sans ça, ce que tu fais n'est pas vraiment un agent, juste un LLM avec du contexte.

Concepts à comprendre

- Function calling : OpenAI peut décider "j'ai besoin d'appeler la fonction `analyse_segment(segment='Premium')`"
- Le boucle agent : LLM → décide d'un outil → appel Python → résultat → LLM → réponse finale
- Tools / Functions / Tool calls — vocabulaire
- Lire la [doc officielle OpenAI Function Calling](#)

Tâches concrètes

Matin (théorique) :

- ☐ Lire la doc OpenAI Function Calling
- ☐ Faire le tutoriel officiel en notebook
- ☐ Lire l'[article Anthropic "Building effective agents"](#)

Après-midi (pratique) :

- ☐ Premier exemple : fonction `get_weather(city)` que tu fais appeler par OpenAI (juste pour comprendre le mécanisme)
- ☐ Comprendre le format JSON Schema des "tools"

Session avec moi

Session #5 — "Function calling expliqué simplement" : on prend le temps de bien comprendre le concept avec un exemple simple, AVANT de l'appliquer à Vector.

Tu finis la journée avec

- Tu comprends ce que c'est qu'un "agent" au sens technique
 - Premier exemple marchant de function calling
 - Tu vois la différence avec ce qu'on faisait jusqu'ici
-

Jour 7 — Mardi : Premier "vrai" outil Vector

Objectif du jour

Vector a maintenant un premier outil qu'il peut décider d'utiliser :

```
analyze_csv_column(column_name, operation).
```

Tâches concrètes

Matin :

- Créer dans `app/tools/` un module `csv_tools.py`
- Implémenter `analyze_column(df, column, operation)` qui calcule mean/median/sum/count selon `operation`
- Définir le JSON Schema du tool pour OpenAI

Après-midi :

- Endpoint `(POST /chat)` qui prend une question, charge un CSV en contexte, et LAISSE Vector décider d'appeler le tool si besoin
- Tests : "Quelle est la moyenne du NPS sur les clients ?" → Vector appelle `analyze_column("nps_score", "mean")` et formule une réponse en français

Session avec moi

Session #6 — "Premier outil agent" : on code ensemble le pattern function calling complet sur ton dataset.

Tu finis la journée avec

- Vector qui appelle des fonctions Python tout seul
 - Comprend la boucle "réflexion → action → résultat → réflexion"
-

Jour 8 — Mercredi : Multi-outils

Objectif du jour

Vector a plusieurs outils et choisit lequel utiliser. C'est là qu'on devient vraiment un agent.

Tâches concrètes

Matin : créer 3 nouveaux outils

- `segment_clients(df, criteria)` : segmentation custom
- `detect_outliers(df, column)` : détection de valeurs aberrantes
- `compare_groups(df, group_col, metric_col)` : comparaison entre groupes

Après-midi : Vector multi-outils

■ Vector reçoit la liste des 4 outils disponibles

■ Il choisit selon la question

■ Tests :

- "Quels segments génèrent le plus de revenu ?" → `segment_clients` + `compare_groups`
- "Y a-t-il des clients avec un comportement anormal ?" → `detect_outliers`
- "Compare le NPS entre les régions" → `compare_groups`

🧡 Session avec moi

Session #7 — "Agent multi-outils" : on conçoit l'architecture pour que Vector puisse choisir intelligemment entre plusieurs tools.

✅ Tu finis la journée avec

- Vector qui orchestre plusieurs outils
 - Tu comprends pourquoi la qualité de la description des outils est critique (le LLM choisit en lisant la description)
-

Jour 9 — Jeudi : Boucle agent multi-step

🎯 Objectif du jour

Vector enchaîne plusieurs appels d'outils dans une seule conversation. Exemple : "analyse les ventes" → il appelle `summary`, puis voit qu'il y a une anomalie → appelle `detect_outliers` sur le bon mois → puis `compare_groups` pour expliquer → puis répond.

🔧 Tâches concrètes

- Implémenter la **boucle agent** : tant qu'OpenAI demande des tool calls, on les exécute et on lui renvoie les résultats, jusqu'à ce qu'il donne une réponse finale
- Limiter à `max_iterations=10` pour éviter les boucles infinies
- Logger chaque étape (très utile pour comprendre ce que fait Vector)
- Renvoyer la réponse finale + la "trace" des outils utilisés

🧡 Session avec moi

Session #8 — "Boucle agent multi-step" : c'est THE pattern central des agents modernes. On la code ensemble avec soin.

✅ Tu finis la journée avec

- Vector qui raisonne sur plusieurs étapes
- Logs visibles de chaque "pensée" de l'agent

- Tu comprends pourquoi LangGraph, OpenAI Agents SDK, etc. existent (ils encapsulent ce pattern)
-

Jour 10 — Vendredi : Tests + documentation + démo

Objectif du jour

Solidifier ce qui a été construit cette semaine. Préparer une démo de mi-stage potentielle.

Tâches concrètes

Matin :

- ☐ Écrire des tests pytest sur les tools (sans appeler OpenAI, juste le code Python pur)
- ☐ Documenter chaque tool avec docstring complète
- ☐ Mettre à jour le README du projet

Après-midi :

- ☐ Préparer un **scénario de démo** : 5 questions qui montrent les capacités de Vector (réutilise les 7 insights cachés du dataset !)
- ☐ Capture vidéo ou screenshots du flow
- ☐ Journal de bord semaine 2

Session avec moi

Session #9 — "Scénario de démo qui claque" : on choisit les questions qui mettent le mieux Vector en valeur, on prépare la mise en scène.

☒ Tu finis la semaine avec

- Un agent multi-outils fonctionnel
 - Tests sur ton code
 - Une démo prête à montrer
-

SEMAINE 3 — Base de données + RAG

Thème : Vector passe d'un script CSV à un système connecté à une vraie base de données. Premiers pas avec les embeddings et pgvector.

Jour 11 — Lundi : Postgres + Docker

Objectif du jour

Postgres tourne en local via Docker. Vector y est connecté.

Concepts à comprendre

- Pourquoi une base de données (vs CSV à chaque requête)
- Docker basics : container, image, volume, port
- Connexion async vs sync

Tâches concrètes

Matin :

- ☐ Lancer Postgres + pgvector via Docker (commande déjà dans le guide setup)
- ☐ Se connecter à la DB avec DBeaver pour valider visuellement
- ☐ Créer le schéma SQL : tables `clients`, `campaigns`, `sales`, `finance` (similaire au dataset)

Après-midi :

- ☐ Installer SQLAlchemy + asyncpg
- ☐ Configurer la connexion dans `app/db.py`
- ☐ Endpoint `GET /clients` qui liste les clients en DB

Session avec moi

Session #10 — "Postgres + SQLAlchemy from scratch" : on monte la DB, le schéma, le premier endpoint qui lit dedans.

Tu finis la journée avec

- Postgres qui tourne en arrière-plan
- Connexion validée
- Premier endpoint qui lit en DB

Jour 12 — Mardi : Charger le dataset synthétique

Objectif du jour

Les 5 CSVs du dataset sont importés en base. Vector peut requêter dessus.

Tâches concrètes

Matin :

- ☐ Script d'import : `scripts/load_dataset.py` qui lit les 5 CSVs et les insère dans les tables Postgres
- ☐ Vérifier les counts (500 clients, 50 campagnes, etc.)

Après-midi :

- Adapter les tools de Vector pour utiliser la DB au lieu des CSVs
- Nouveau tool : `query_database(sql_query)` — Vector peut écrire du SQL ⚠️ (avec garde-fous : SELECT only)
- Test : "Combien de ventes en septembre 2025 ?" → Vector génère le SQL et exécute

🧡 Session avec moi

Session #11 — "Vector qui parle SQL" : on construit le tool SQL avec les bonnes sécurités (pas de DROP, INSERT, etc. depuis le LLM).

✅ Tu finis la journée avec

- Toutes les données en base
 - Vector qui peut générer et exécuter du SQL
 - Tu comprends les enjeux de sécurité (SQL injection via LLM)
-

Jour 13 — Mercredi : Embeddings & pgvector

🎯 Objectif du jour

Comprendre les embeddings et premier exemple de recherche sémantique.

📚 Concepts à comprendre

- Embedding = vecteur représentant le sens d'un texte
- Distance cosinus / euclidienne
- À quoi ça sert (recherche par similarité, RAG)

🛠️ Tâches concrètes

Matin :

- Activer l'extension pgvector dans Postgres
- Ajouter une colonne `embedding VECTOR(1536)` à une table de test
- Comprendre l'API OpenAI Embeddings (`text-embedding-3-small`)

Après-midi :

- Petit exemple : indexer 20 textes de descriptions de produits, faire une recherche par similarité
- Visualiser les résultats dans DBeaver

🧡 Session avec moi

Session #12 — "Embeddings & recherche vectorielle" : on démystifie les vecteurs, on fait une démo claire de recherche sémantique.

✓ Tu finis la journée avec

- Tu comprends conceptuellement les embeddings
 - Premier exemple de recherche sémantique qui marche
 - Vue d'ensemble du RAG
-

Jour 14 — Jeudi : RAG pour Vector

🎯 Objectif du jour

Vector se souvient de ses analyses passées et peut y faire référence.

🔧 Tâches concrètes

Matin :

- ❑ Créer table `analysis_history` : chaque analyse de Vector est stockée avec son embedding
- ❑ À chaque nouvelle question, Vector récupère les 3 analyses les plus similaires des semaines précédentes
- ❑ Les inclure dans son contexte

Après-midi :

- ❑ Tester : pose une question similaire à une analyse passée → Vector doit pouvoir y faire référence
- ❑ Endpoint `GET /analysis/history` qui liste les analyses passées
- ❑ Endpoint `GET /analysis/search?query=...` pour faire de la recherche sémantique

💛 Session avec moi

Session #13 — "RAG appliqué à Vector" : on architecture la mémoire de Vector via pgvector.

✓ Tu finis la journée avec

- Vector a une mémoire à long terme
 - Recherche sémantique fonctionnelle
 - Premier vrai pattern RAG en prod
-

Jour 15 — Vendredi : Migrations + qualité + journal

🎯 Objectif du jour

Tout est propre, versionné, documenté.

Tâches concrètes

Matin :

- ☐ Installer Alembic et faire ta première migration
- ☐ Comprendre pourquoi les migrations sont critiques (vs créer les tables à la main)

Après-midi :

- ☐ Refactoriser le code DB pour utiliser des "repositories" (pattern)
- ☐ Tests d'intégration sur les endpoints DB
- ☐ Journal de bord semaine 3
- ☐ Démo intermédiaire à ton encadrant si pertinent

Session avec moi

Session #14 — "Architecture DB pro" : on revoit la structure et on applique les bonnes pratiques.

☒ Tu finis la semaine avec

- Postgres + pgvector maîtrisés
 - Vector qui requête une vraie DB
 - Mémoire long terme via RAG
 - Code DB propre et versionné
-

SEMAINE 4 — Détection d'insights + démo finale

Thème : Vector devient vraiment intéressant — il découvre les patterns que tu lui as cachés. Préparation d'une démo qui claque.

Jour 16 — Lundi : Outils de détection avancée

Objectif du jour

Vector a maintenant des outils statistiques pointus pour détecter les opportunités (cf. feature #2 du site AI Commandos).

Tâches concrètes

Matin : nouveaux tools

- ☐ `detect_anomalies(df, column, method)` : z-score, IQR, isolation forest
- ☐ `find_correlations(df, target)` : trouve les colonnes corrélées
- ☐ `seasonality_analysis(df, date_col, value_col)` : détecte les patterns saisonniers
- ☐ `cohort_analysis(df, cohort_col, metric_col)` : analyse de cohortes

Après-midi :

- Tests sur le dataset synthétique
- Premier insight automatique : *"Trouve une anomalie dans les ventes"* → Vector doit détecter la saisonnalité Q4 vs été

🧡 Session avec moi

Session #15 — "Outils analytics avancés" : on code ensemble les 4 tools, en s'assurant qu'ils sont bien décrits pour le LLM.

✅ Tu finis la journée avec

- Vector capable de faire de l'analyse statistique sophistiquée
 - Premier insight automatique détecté
-

Jour 17 — Mardi : Détection des 7 insights cachés

🎯 Objectif du jour

Vector trouve **tous** les insights cachés du dataset, en autonomie.

🔧 Tâches concrètes

Matin : meta-prompt d'exploration

■ Endpoint (`POST /explore`) : Vector explore librement les données et remonte ce qu'il trouve d'intéressant

■ Tester sur les 7 insights documentés (cf. README du dataset) :

1. Paradoxe Manufacturing (NPS bas + LTV haute)
2. Churn risk des Starters
3. ROI caché des Events
4. Mismatch LinkedIn × Santé
5. LTV Manufacturing
6. Profil Champion = upsell
7. Marché caché PACA

Après-midi :

- Améliorer les prompts pour que Vector trouve au moins 4-5 des 7
- Documenter ce qu'il rate et pourquoi

🧡 Session avec moi

Session #16 — "Faire détecter des insights par un agent" : on travaille les prompts d'exploration et les patterns analytiques.

✅ Tu finis la journée avec

- Vector qui découvre tout seul plusieurs insights pertinents
 - Compréhension fine de comment guider un LLM vers la découverte
-

Jour 18 — Mercredi : Génération de recommandations

🎯 Objectif du jour

Pour chaque insight détecté, Vector génère 2-3 recommandations actionables (cf. feature #4 du site AI Commandos).

🔧 Tâches concrètes

Matin :

- ▣ Pydantic model `Recommendation` : titre, description, impact estimé, priorité, ressources nécessaires
- ▣ Vector génère systématiquement 2-3 recos par insight détecté

Après-midi :

- ▣ Endpoint `POST /full-analysis` qui enchaîne :
 1. Explore les données
 2. Détecte les insights
 3. Génère les recommandations
 4. Renvoie le tout dans un rapport JSON structuré

💛 Session avec moi

Session #17 — "Recommandations data-driven" : on définit ensemble le format des recos pour qu'elles soient vraiment utiles.

✅ Tu finis la journée avec

- Vector qui produit des recos en français, business-savvy
 - Rapport complet automatique
-

Jour 19 — Jeudi : Front simple OU API polishing

🎯 Objectif du jour

À adapter selon ce qu'a dit Welyne au kick-off.

Option A — Frontend dans ton scope (Next.js dashboards)

- ☐ Créer un projet Next.js minimal qui affiche les analyses
- ☐ Page d'upload de CSV
- ☐ Affichage des dashboards avec Recharts ou Tremor
- ☐ Connexion à ton API FastAPI

Option B — Frontend hors scope (polish API)

- ☐ Authentification API (clé API simple)
- ☐ Rate limiting (slowapi)
- ☐ Logs structurés (structlog)
- ☐ Documentation OpenAPI enrichie
- ☐ Containerisation Docker

💛 Session avec moi

Session #18 — "Frontend ou polish API" : on prend ensemble la décision selon ce que Welyne attend, puis on code.

✅ Tu finis la journée avec

- Soit un mini-frontend qui affiche les insights
 - Soit une API prête pour la prod
-

Jour 20 — Vendredi : Démo + journal + retro

🎯 Objectif du jour

Bilan du premier mois + démo prête à montrer à Welyne et AI Commandos.

🔧 Tâches concrètes

Matin : démo finale

- ☐ Préparer un scénario de démo de 10 minutes
- ☐ Captures vidéo + screenshots
- ☐ Préparer la slide deck ou le doc qui résume le travail

Après-midi : rétrospective + journal

- ☐ Journal de bord semaine 4 + bilan mois 1
- ☐ Ce qui a marché / ce qui a moins marché
- ☐ Ce qui reste à faire pour le mois 2
- ☐ Présenter à ton encadrant la démo

💛 Session avec moi

Session #19 — "Préparation démo finale" : on prépare la mise en scène, les transitions, les questions probables.

Tu finis le mois avec

- ☒ Un Vector POC fonctionnel et impressionnant
 - ☒ FastAPI maîtrisé
 - ☒ OpenAI function calling maîtrisé
 - ☒ Postgres + pgvector + RAG basique maîtrisé
 - ☒ Boucle agent multi-step maîtrisée
 - ☒ Détection d'insights automatique fonctionnelle
 - ☒ Un repo GitHub propre avec ~20 commits
 - ☒ Un journal de bord détaillé pour ton rapport de stage
 - ☒ Une démo qui claque
-

Ce qui reste pour le mois 2

Le mois 2 dépendra de :

1. **Le feedback de Welyne et AI Commandos** sur ta démo
2. **Le scope production** qu'ils veulent vraiment livrer
3. **L'intégration avec CORE** (l'orchestrateur)
4. **Les vraies données client** (si fournies)
5. **Le déploiement** (Docker + cloud)

On replannifiera ensemble en début de mois 2 avec ces inputs.

Aide-mémoire quotidien

Routine du matin (5 min)

- ☐ Café ☕
- ☐ Ouvre ton journal de la veille, relis tes notes
- ☐ Fixe-toi 1-3 objectifs concrets pour la journée

Routine du soir (10 min)

- ☐ Commit ce qui n'est pas commité
- ☐ Push sur ton repo
- ☐ Écris dans ton journal :
 -  Ce que j'ai fait aujourd'hui

- 🎓 Ce que j'ai appris
- 😬 Ce qui m'a bloquée et comment résolu
- ❓ Question(s) à poser demain

Quand tu coinces (max 1h seule)

1. Lire l'erreur attentivement (Python te dit souvent la solution)
2. Chercher l'erreur sur Google + Stack Overflow
3. Demander à un collègue chez Welyne
4. M'envoyer un message ici

Quand tu réussis quelque chose

- 🎉 Note-le dans ton journal !
 - C'est ce qui fera ton rapport de stage tellement plus riche
 - C'est ce qui te rappellera ta progression dans les jours difficiles
-

Maintenant, plus qu'à attaquer

Tu as un plan. Tu as les outils. Tu as l'environnement. **Tu as TOUT ce qu'il faut** pour réussir ce mois 1.

Le secret n'est pas la perfection — c'est la **constance** : un peu chaque jour, en suivant le plan, en ajustant quand nécessaire.

Le premier jour de stage, ouvre-moi ce chat avec :

"Stage commencé. Premier message de mon encadrant : [...]. Je continue le plan jour 1 ou on adapte ?"

Et on attaque ensemble. 💪